Classification de la qualité du vin

Rapport technique – Projet de Machine Learning 1

Rayane Ait Braham Tammy Benmimoun Ahmed Hammadi

EFREI Paris – ING2 BDML – 2026-2027

Résumé

Ce rapport décrit la démarche suivie pour prédire si un vin est « bon » (note de qualité supérieure ou égale à 7) à partir de onze mesures physico-chimiques, sur les données *Wine Quality* du vinho verde. Nous détaillons le nettoyage des données, l'analyse exploratoire, la sélection de variables, la comparaison de six modèles, le réglage des hyperparamètres par Grid Search et Random Search, le choix du seuil de décision puis l'évaluation finale sur un jeu de test tenu à l'écart. Tout le code est dans le notebook `projet_vin_ml_simple.ipynb`.

Table des matières

1 Introduction **3**

2 Chargement et nettoyage des données **3**

2.1 Chargement et fusion 3

2.2 Valeurs manquantes et doublons 3

2.3 Valeurs aberrantes 3

3 Cible binaire et équilibre des classes **3**

4 Analyse exploratoire **4**

4.1 Corrélations 4

4.2 Distributions et boxplots 5

5 Découpage des données et validation **5**

6 Sélection de variables **6**

7 Modèles étudiés **6**

8 Évaluation initiale et biais/variance **7**

8.1 Métriques 7

8.2 Résultats en validation croisée 7

8.3 Compromis biais/variance 8

9 Optimisation des hyperparamètres **8**

10 Choix du seuil de décision **10**

11 Évaluation finale sur le jeu de test **10**

12 Interprétation du modèle **11**

13 Sauvegarde et application **11**

14 Limites et conclusion **12**

1 Introduction

La qualité d'un vin dépend de nombreux paramètres chimiques (acidité, sucre, alcool, sulfites…). Le sujet demande de construire un classifieur supervisé qui sépare les bons vins des autres à partir de ces mesures. Nous avons retenu la version binaire du problème : un vin est positif (classe 1) si sa note vaut au moins 7.

Le jeu de données vient de Cortez et al. (2009). Il contient 1599 vins rouges et 4898 vins blancs, chacun décrit par 11 mesures et une note entre 0 et 10 attribuée par des dégustateurs. Les deux fichiers CSV utilisent le point-virgule comme séparateur.

Le rapport suit l'ordre du notebook. Pour chaque étape nous expliquons ce qui a été fait, puis pourquoi nous l'avons fait ainsi.

2 Chargement et nettoyage des données

2.1 Chargement et fusion

Les deux fichiers sont lus avec `pandas` puis concaténés en un seul tableau de 6497 lignes. Comme les vins rouges et blancs n'ont pas le même profil chimique, nous ajoutons une variable `type` (0 pour rouge, 1 pour blanc). C'est la seule variable catégorielle du jeu de données, et un codage binaire suffit : il n'y a pas besoin de *one-hot encoding* avec seulement deux modalités.

2.2 Valeurs manquantes et doublons

Aucune valeur manquante n'a été trouvée, il n'y a donc rien à imputer. En revanche, 1177 lignes sont des doublons exacts (environ 18 % du total). Nous les avons supprimés, ce qui laisse 5320 vins. La raison est pratique : si une même ligne se retrouve à la fois dans le train et dans le test, le score de test est trop optimiste puisque le modèle a déjà vu l'exemple.

2.3 Valeurs aberrantes

Nous avons compté, pour chacune des 11 mesures, les points situés au-delà de $1{,}5 \times \mathrm{IQR}$ des quartiles, et calculé l'asymétrie de chaque distribution. Les variables les plus touchées sont l'acidité fixe (304 points hors bornes), l'acidité volatile (279) et les chlorures (237), ces derniers ayant l'asymétrie la plus forte (5,34).

Nous avons choisi de **conserver** ces points. Un vin très sucré ou très riche en sulfites est un vin qui existe vraiment, pas une erreur de saisie, et les supprimer reviendrait à retirer justement des cas atypiques que le modèle devra rencontrer. Leur influence est limitée d'une autre façon : standardisation des variables, et usage de modèles peu sensibles aux valeurs extrêmes (arbres, forêts).

3 Cible binaire et équilibre des classes

La variable cible est définie par good $= 1$ si quality ≥ 7, et 0 sinon. Les bons vins ne représentent qu'environ 19 % de l'échantillon, le jeu de données est donc clairement déséquilibré.

Classe	Effectif	Proportion
Pas bon (0)	4311	81 %
Bon (1)	1009	19 %

TABLE 1 – Répartition des classes après suppression des doublons.

La part de bons vins est de 20,8 % chez les blancs contre 13,5 % chez les rouges. Conséquence importante : un modèle qui répondrait toujours « pas bon » aurait déjà une accuracy d'environ 81 %. L'accuracy seule ne permet donc pas de juger un modèle ici. Nous nous appuyons surtout sur la précision, le rappel, le F1-score et l'AUC. Pour tenir compte du déséquilibre, les découpages sont stratifiés et nous utilisons `class_weight="balanced"` quand le modèle le permet (ce n'est pas le cas de k-NN ni de Naive Bayes).

4 Analyse exploratoire

4.1 Corrélations

La matrice de corrélation (figure 1) montre que l'alcool est la variable la plus liée à la note (+0,47). Viennent ensuite la densité (−0,33), l'acidité volatile (−0,27) et les chlorures (−0,20). Les autres corrélations avec la note sont faibles (en valeur absolue, 0,11 ou moins).

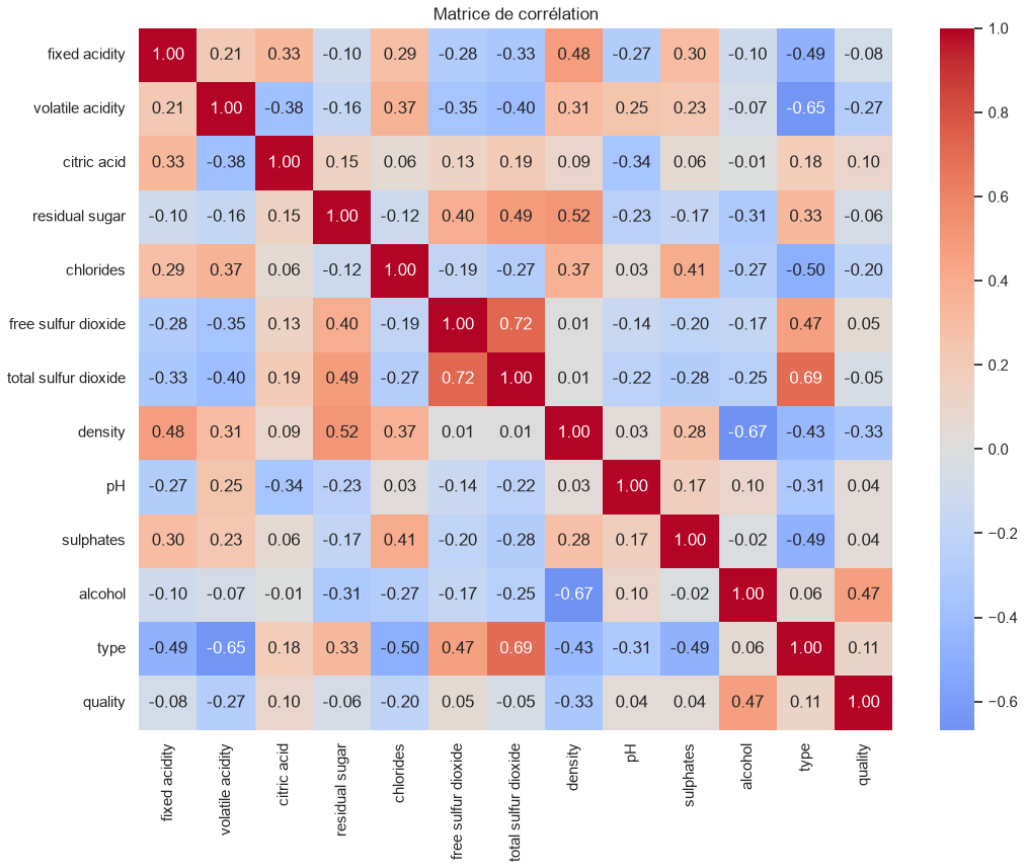


FIGURE 1 – Matrice de corrélation des variables et de la note.

Certaines variables explicatives sont fortement corrélées entre elles : dioxyde de soufre libre

et total (0,72), densité et alcool (0,67), sucre résiduel et densité (0,52). Cela pose deux problèmes. D'abord, ces variables apportent une information redondante, ce qui justifie une étape de sélection. Ensuite, cela contredit l'hypothèse d'indépendance de Naive Bayes, dont on peut attendre des résultats plus faibles.

4.2 Distributions et boxplots

Les histogrammes montrent des distributions souvent asymétriques, avec des queues longues à droite. Les boxplots comparant bons et autres vins (figure 2) confirment que l'alcool est plus élevé chez les bons vins, alors que la densité et l'acidité volatile y sont plus basses.

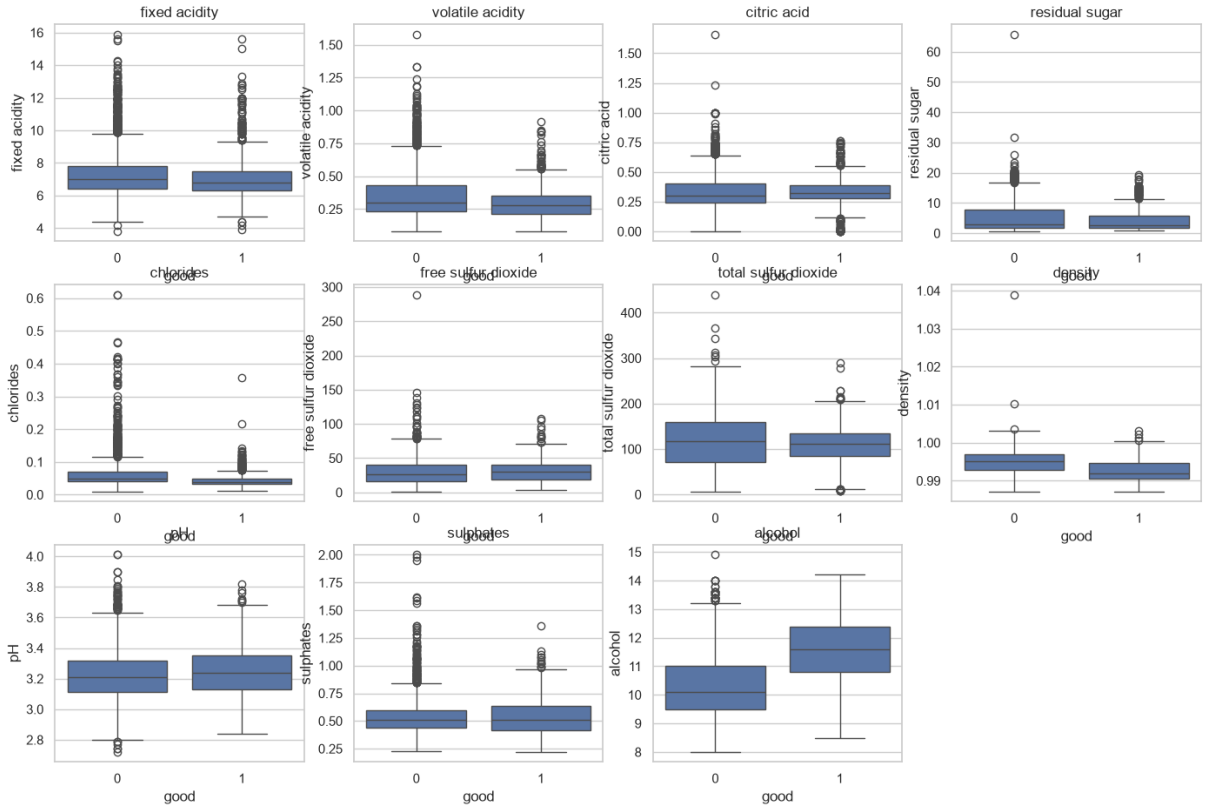


FIGURE 2 – Boxplots de chaque mesure selon la classe (bon / pas bon).

5 Découpage des données et validation

Les données sont séparées en 80 % d'entraînement (4256 vins) et 20 % de test (1064 vins), avec stratification sur la cible pour garder la même proportion de bons vins dans les deux ensembles (`random_state=42` pour la reproductibilité). Le jeu de test n'est utilisé qu'une seule fois, à la toute fin.

Pour tout le reste (choix du modèle, réglage, seuil), nous utilisons une validation croisée stratifiée à 5 plis sur le train uniquement.

Un point nous a demandé de l'attention : la standardisation et la sélection de variables sont intégrées dans un `Pipeline` scikit-learn. Elles sont ainsi recalculées dans chaque pli, uniquement sur la partie d'entraînement du pli. Si on les calculait avant la validation croisée sur tout le

train, des informations du pli de validation influenceraient l'entraînement (fuite de données) et les scores seraient trop favorables.

6 Sélection de variables

Nous avons comparé trois familles de méthodes, toutes calculées sur le train seulement.

Famille	Méthode	Critère
Filtre	Test F (ANOVA), information mutuelle	score par variable
Wrapper	RFE avec régression logistique	rang d'élimination
Embedded	Random Forest	importance des variables

Table 2 – Méthodes de sélection de variables utilisées.

Pour synthétiser, nous avons classé les variables selon chaque critère puis calculé leur rang moyen. L'alcool arrive premier pour les quatre critères, suivi de la densité et de l'acidité volatile (tableau 3). À l'inverse, le dioxyde de soufre libre et le type de vin sont en bas du classement. Le type reçoit pourtant un score F non négligeable ; sa faible importance dans la forêt et l'information mutuelle nulle montrent que son effet passe surtout par les autres mesures.

Variable	Score F	Info. mutuelle	Rang RFE	Importance RF	Rang moyen
alcohol	924,9	0,099	1	0,198	1,00
density	413,6	0,058	5	0,123	2,75
volatile acidity	88,5	0,026	2	0,085	3,50
chlorides	109,9	0,048	10	0,089	4,75
total sulfur dioxide	23,5	0,024	8	0,082	6,50
…					
free sulfur dioxide	0,1	0,005	9	0,070	10,25
type	27,8	0,000	11	0,003	10,25

Table 3 – Classement des variables (extrait : cinq premières et deux dernières).

Pour choisir le nombre k de variables à garder, nous avons évalué une régression logistique avec `SelectKBest` pour k allant de 3 à 12, en validation croisée. Le F1 passe de 0,512 pour $k = 3$ à 0,533 pour $k = 12$, en augmentant lentement : aucune variable ne gêne vraiment le modèle et le gain de la sélection est faible. Avec seulement 12 variables au départ, elle sert donc surtout à comprendre les données. Par défaut nous prenons $k = 8$ pour la comparaison initiale, puis k est traité comme un hyperparamètre ($k \in \{8, 12\}$) lors de l'optimisation.

7 Modèles étudiés

Nous avons comparé les cinq modèles demandés par le sujet, plus une Random Forest. Le tableau 4 résume pourquoi chacun nous a semblé pertinent, ou non, sur ces données.

Modèle	Nature	Remarque pour ce problème
Régression logistique	linéaire	Simple et interprétable, mais ne capture pas une frontière non linéaire.
k-NN	non linéaire, par distance	Exige des variables standardisées, sensible au bruit et au déséquilibre.
Arbre de décision	non linéaire	Règles lisibles et robuste aux valeurs extrêmes, mais variance élevée.
Naive Bayes (gaussien)	probabiliste	Suppose les variables indépendantes, ce qui est faux ici (cf. corrélations).
SVM, noyau RBF	non linéaire	Adapté à des variables continues et bruitées, sensible à C et γ.
Random Forest	ensemble d'arbres	Réduit la variance de l'arbre seul, fournit l'importance des variables.

TABLE 4 – Modèles comparés et justification du choix.

Chaque modèle est placé dans le même `Pipeline` : standardisation, sélection de k variables par test F, puis classifieur. Ainsi la comparaison ne porte que sur le classifieur.

8 Évaluation initiale et biais/variance

8.1 Métriques

Pour une classe positive « bon vin », avec VP, FP, VN, FN les effectifs de la matrice de confusion :

$$\text{précision} = \frac{VP}{VP+FP}, \qquad \text{rappel} = \frac{VP}{VP+FN}, \qquad F_1 = 2\,\frac{\text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}}.$$

La précision indique quelle part des vins annoncés bons le sont réellement, le rappel quelle part des vrais bons vins est retrouvée. L'AUC mesure la capacité du modèle à classer un bon vin au-dessus d'un autre, indépendamment du seuil.

8.2 Résultats en validation croisée

Chaque modèle est évalué en validation croisée à 5 plis sur le train, avec les paramètres par défaut (tableau 5). Nous relevons aussi le F1 sur les plis d'entraînement, pour mesurer l'écart avec le F1 de validation (colonne F1 train).

Modèle	Accuracy	Précision	Rappel	F1	AUC	F1 train
Régression logistique	0,735	0,399	0,783	0,528	0,816	0,527
k-NN	0,816	0,520	0,405	0,455	0,783	0,618
Arbre	0,783	0,426	0,408	0,416	0,639	1,000
Naive Bayes	0,742	0,394	0,665	0,495	0,769	0,490
SVM	0,721	0,387	0,803	0,522	0,820	0,540
Random Forest	0,813	0,508	0,555	0,530	0,830	0,993

TABLE 5 – Scores moyens en validation croisée (paramètres par défaut).

Plusieurs constats se dégagent. L'accuracy de k-NN, de la forêt et de l'arbre (autour de 0,8) est proche de la référence de 0,81, elle ne dit donc pas grand-chose. La régression logistique et

le SVM ont un rappel élevé (autour de 0,78 à 0,80) mais une précision basse (environ 0,39) : la pondération des classes les pousse à annoncer beaucoup de bons vins. À l'inverse, k-NN a la meilleure précision (0,52) mais retrouve moins de 41 % des bons vins. La Random Forest obtient le meilleur F1 (0,530) et la meilleure AUC (0,830), très près de la régression logistique en F1. L'arbre est nettement le plus faible (AUC de 0,639).

8.3 Compromis biais/variance

Nous avons tracé des courbes d'apprentissage (F1 en fonction de la taille du train) pour la régression logistique, l'arbre et la Random Forest. La lecture est la suivante : un F1 très haut en entraînement mais nettement plus bas en validation indique une variance élevée (surapprentissage), alors que des scores bas et proches indiquent un biais élevé.

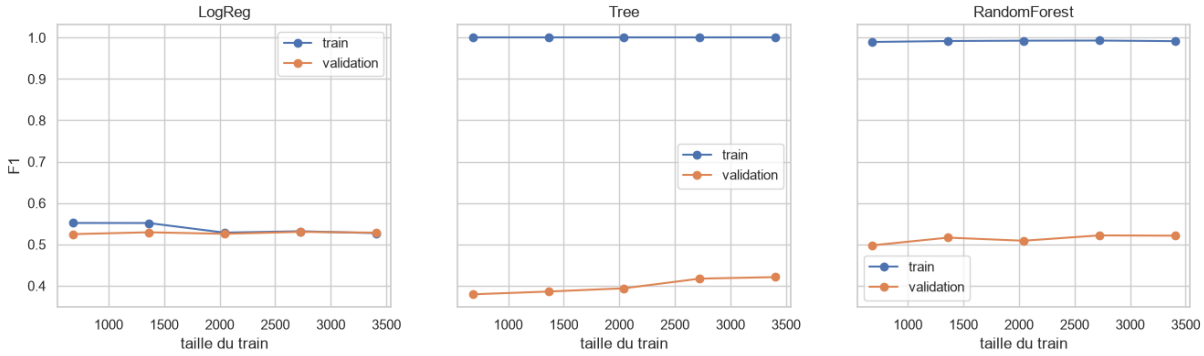


FIGURE 3 – Courbes d'apprentissage de trois modèles.

Sur la figure, la régression logistique a un F1 d'entraînement et de validation presque confondus, autour de 0,53, et ces courbes ne bougent plus quand on ajoute des données : le modèle est limité par sa forme linéaire (biais élevé) et plus de données n'y changeraient rien. L'arbre non contraint est à l'opposé : F1 d'entraînement égal à 1 et F1 de validation entre 0,38 et 0,42, c'est un surapprentissage net (variance élevée). La Random Forest a aussi un F1 d'entraînement proche de 1 (0,99) pour une validation autour de 0,52 ; l'écart reste grand mais la validation est bien meilleure que celle de l'arbre, ce qui illustre l'intérêt de moyenner de nombreux arbres. Sa courbe de validation monte légèrement avec la taille du train, donc davantage de données l'aiderait un peu.

9 Optimisation des hyperparamètres

Le critère optimisé est le F1, toujours en validation croisée à 5 plis sur le train. Nous avons utilisé les deux techniques du sujet.

Grid Search. Recherche exhaustive sur une grille, appliquée à la régression logistique, au k-NN, à l'arbre, à Naive Bayes et au SVM. Le tableau 6 donne les valeurs testées.

Modèle	Hyperparamètres testés
Régression logistique	$C \in \{0{,}01;\ 0{,}1;\ 1;\ 10\}$
k-NN	*n_neighbors* $\in \{3, 5, 11, 21, 31\}$, poids uniformes ou par distance
Arbre	profondeur max $\in \{3, 5, 8, 12, \text{aucune}\}$, `min_samples_leaf` $\in \{1, 5, 10, 20\}$
Naive Bayes	`var_smoothing` $\in \{10^{-9}, 10^{-7}, 10^{-5}, 10^{-3}, 10^{-1}\}$
SVM	$C \in \{0{,}1;\ 1;\ 10\}$, $\gamma \in \{\text{scale};\ 0{,}1\}$

TABLE 6 – Grilles utilisées (en plus de $k \in \{8, 12\}$ variables sélectionnées).

Random Search. Tirage aléatoire de combinaisons, appliqué au SVM (12 tirages, C entre 0,1 et 100 et γ entre 10^{-3} et 1, tirés selon une loi log-uniforme) et à la Random Forest (10 tirages sur le nombre d'arbres, la profondeur et `min_samples_leaf`). Pour la forêt, une grille complète aurait été trop longue à parcourir, ce qui est le cas typique où le Random Search est utile.

Le SVM a été traité avec les deux méthodes, ce qui permet de comparer le score atteint et le temps de calcul : la grille donne 0,546 en 10,2 s, le Random Search 0,539 en 8,6 s. Le résultat est très proche et le temps du même ordre, car la grille du SVM est petite. L'avantage du Random Search apparaît surtout sur de grands espaces, comme pour la forêt.

Modèle	Méthode	F1 (CV)	Temps (s)	Meilleurs paramètres
Régression logistique	Grid	0,539	0,6	$C = 0{,}01$, $k = 12$
k-NN	Grid	0,492	1,9	5 voisins, poids par distance, $k = 12$
Arbre	Grid	0,511	2,7	profondeur 3, `min_samples_leaf` 20, $k = 8$
Naive Bayes	Grid	0,499	0,5	`var_smoothing` 0,1, $k = 12$
SVM	Grid	0,546	10,2	$C = 1$, $\gamma = 0{,}1$, $k = 12$
SVM	Random	0,539	8,6	$C \approx 83$, $\gamma \approx 0{,}005$, $k = 12$
Random Forest	Random	0,556	11,7	100 arbres, profondeur 20, `min_samples_leaf` 1, $k = 12$

TABLE 7 – Résultats des recherches d'hyperparamètres (F1 en validation croisée).

Les modèles optimisés sont ensuite réévalués avec les mêmes métriques qu'au tableau 5 (tableau 8). Le modèle final est celui qui a le meilleur F1 en validation croisée : la Random Forest issue du Random Search.

Modèle	Accuracy	Précision	Rappel	F1	AUC	F1 train
Random Forest (random)	0,824	0,534	0,581	0,556	0,847	0,987
SVM (grid)	0,748	0,415	0,799	0,546	0,843	0,590
Régression logistique (grid)	0,742	0,408	0,796	0,539	0,828	0,540
SVM (random)	0,732	0,400	0,828	0,539	0,841	0,558
Arbre (grid)	0,723	0,384	0,762	0,511	0,798	0,528
Naive Bayes (grid)	0,748	0,400	0,660	0,499	0,777	0,496
k-NN (grid)	0,829	0,566	0,436	0,492	0,794	1,000

TABLE 8 – Modèles optimisés, validation croisée à 5 plis sur le train.

Le réglage améliore tous les modèles, surtout l'arbre dont le F1 passe de 0,416 à 0,511 en limitant sa profondeur à 3, ce qui réduit fortement son surapprentissage (F1 d'entraînement

de 1,000 à 0,528). La Random Forest reste la meilleure en F1 et en AUC, mais l'écart avec le SVM et la régression logistique est faible (environ 0,01 à 0,02 de F1). Le k-NN optimisé a un F1 d'entraînement de 1,000 parce que les poids par distance font que chaque point d'entraînement se retrouve lui-même comme voisin ; ce chiffre ne doit pas se lire comme un surapprentissage au même titre que pour l'arbre.

10 Choix du seuil de décision

Par défaut, un vin est classé « bon » si son score dépasse 0,5 (ou 0 pour la fonction de décision d'un SVM). Avec une classe rare, ce seuil n'est pas forcément le meilleur pour le F1.

Pour le choisir sans toucher au test, nous calculons avec `cross_val_predict` un score « hors pli » pour chaque vin du train : il est noté par un modèle qui ne l'a pas vu pendant son entraînement. On parcourt ensuite la courbe précision-rappel et on garde le seuil qui maximise le F1. On obtient un seuil de 0,428 (un peu plus bas que 0,5, donc plus généreux avec la classe « bon ») pour un F1 de 0,563 en validation croisée, contre 0,556 avec le seuil par défaut. Le gain est faible.

11 Évaluation finale sur le jeu de test

C'est la seule fois où le jeu de test est utilisé. Nous comparons le seuil par défaut et le seuil optimisé.

	Précision	Rappel	F1	AUC
Seuil par défaut	0,56	0,61	0,58	0,875
Seuil optimisé (0,428)	0,52	0,74	0,61	–

TABLE 9 – Résultats du modèle final sur le jeu de test (classe « bon vin »).

Sur le test, le F1 de la classe « bon » est de 0,58 au seuil par défaut, cohérent avec les 0,556 observés en validation croisée : il n'y a pas de mauvaise surprise. Le seuil abaissé échange de la précision (0,56 vers 0,52) contre du rappel (0,61 vers 0,74) et gagne 0,03 de F1. L'accuracy est de 0,83, soit à peine au-dessus de la référence de 0,81, ce qui confirme qu'il faut la lire avec méfiance. L'AUC de 0,875 montre que le modèle classe plutôt bien les vins malgré un F1 modeste.

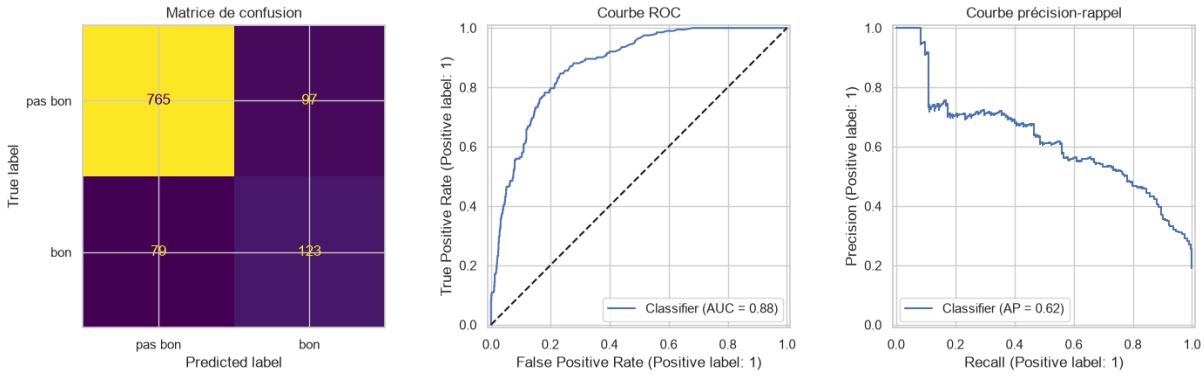


FIGURE 4 – Évaluation finale : matrice de confusion, courbe ROC et courbe précision-rappel.

Nous avons aussi calculé les scores séparément pour les vins rouges et les vins blancs, car les deux populations n'ont pas la même proportion de bons vins ni les mêmes effectifs dans le test.

Type	Vins	Bons vins	Précision	Rappel	F1
Rouge	275	37	0,568	0,568	0,568
Blanc	789	165	0,557	0,618	0,586

TABLE 10 – Résultats du modèle final par type de vin.

Les performances sont proches entre les deux types, un seul modèle suffit donc. Attention toutefois : il n'y a que 37 bons vins rouges dans le test, l'écart observé n'est pas significatif.

Le choix entre précision et rappel dépend de l'usage. Pour repérer le maximum de bons vins, on privilégie le rappel ; pour éviter d'annoncer à tort un grand cru, on privilégie la précision. Le seuil sert à régler ce compromis.

12 Interprétation du modèle

Nous avons utilisé deux outils complémentaires.

Importance par permutation. On mélange aléatoirement une variable du jeu de test et on mesure la baisse de F1 (15 répétitions). L'alcool domine nettement : sa permutation fait perdre 0,244 de F1. Viennent ensuite la densité (0,101), les chlorures (0,083) et le dioxyde de soufre total (0,052) ; toutes les autres variables sont à 0,035 ou moins, et le type et l'acidité fixe sont proches de zéro.

Coefficients de la régression logistique. Les variables étant standardisées, les coefficients sont comparables entre eux. Un coefficient positif augmente la probabilité d'être un bon vin, un coefficient négatif la diminue. La densité ($-1{,}37$), l'acidité volatile ($-0{,}62$) et le dioxyde de soufre total ($-0{,}43$) jouent contre la qualité ; l'alcool ($+0{,}62$) joue pour, avec le sucre résiduel ($+0{,}98$), l'acidité fixe ($+0{,}78$) et le pH ($+0{,}52$).

Ces coefficients se lisent avec prudence. Le sucre résiduel a par exemple un coefficient fortement positif alors que sa corrélation simple avec la note est quasi nulle ($-0{,}06$). La densité dépend de l'alcool et du sucre résiduel : une fois la densité (très négative) prise en compte, le modèle utilise le sucre pour compenser. Quand les variables sont corrélées, l'effet est réparti entre elles et un coefficient seul peut être trompeur. Le classement par permutation est plus stable pour juger de l'importance globale.

13 Sauvegarde et application

Le modèle final, la liste des variables, le seuil et le nom du modèle sont enregistrés dans un fichier `model_vin.joblib`. Nous le rechargeons dans le notebook pour vérifier qu'il prédit correctement sur un vin du jeu de test. Ce fichier est prévu pour être utilisé dans une application web Streamlit (`app.py`), qui fait partie des bonus et sera réalisée dans un second temps.

14 Limites et conclusion

Limites. La note de qualité est subjective : elle vient de dégustateurs et deux personnes peuvent noter différemment le même vin. De plus, onze mesures chimiques ne contiennent pas toute l'information qui fait un bon vin (cépage, région, millésime, élevage). Il est donc normal qu'aucun modèle ne soit parfait. Par ailleurs, la classe « bon » est peu représentée, ce qui rend les scores sur le test assez variables d'un découpage à l'autre.

Conclusion. Après suppression de 1177 doublons, nous avons travaillé sur 5320 vins dont seulement 19 % sont bons. Le déséquilibre nous a poussés à ne pas retenir l'accuracy comme critère et à choisir le F1 et l'AUC. Grâce au `Pipeline`, la validation croisée est exempte de fuite de données, et le jeu de test n'a servi qu'une fois. L'alcool ressort comme le facteur le plus déterminant de façon cohérente avec toutes les méthodes (corrélation, sélection de variables, permutation). Le modèle retenu est la Random Forest réglée par Random Search, qui obtient sur le test un F1 de 0,58 pour la classe « bon » avec le seuil par défaut (0,61 avec le seuil optimisé) et une AUC de 0,875.

Référence

P. Cortez, A. Cerdeira, F. Almeida, T. Matos, J. Reis. *Modeling wine preferences by data mining from physicochemical properties*. Decision Support Systems, 47(4), 2009.